

Ejercicio 3 - Análisis

Sistema de Control de Órdenes de Servicio de un Taller Automotriz

1. ¿Qué clases son necesarias para representar el problema planteado?

Se utilizarán cuatro clases: **Order**, que representa cada orden de servicio; **OrdersController**, que administra la colección de órdenes y las operaciones realizadas sobre ella; **View**, que obtiene, valida y muestra la información al usuario; y **Main**, que contiene el método `main()` y controla el flujo general del programa.

2. ¿Qué propiedades y métodos tendrá cada clase?

Order tendrá `orderNumber`, `ownerName`, `vehicleLicensePlate`, `serviceDescription` y `estimatedCost`, junto con su constructor, getters y setters para la descripción y el costo. **OrdersController** tendrá una `List<Order> orders` y métodos para agregar, obtener, buscar, modificar, cancelar, buscar por placa, calcular total, promedio, mayor costo y cantidad de órdenes. **View** tendrá una referencia a `OrdersController` y un `Scanner`, además de métodos para solicitar los datos, mostrar el menú, órdenes, resultados, mensajes y excepciones. **Main** tendrá únicamente `main()`.

3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?

Los números de orden serán **int**, los nombres, placas y descripciones serán **String**, y los costos serán **double**. La colección será **List<Order>**. Los métodos devolverán `void` cuando únicamente realicen una acción, `Order` cuando devuelvan una orden, `List<Order>` cuando puedan devolver varias órdenes, `int` para cantidades u opciones y `double` para resultados de costos.

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección dinámica almacenará objetos **Order**. Cada objeto contiene el número de orden, nombre del propietario, placa del vehículo, descripción del servicio y costo estimado.

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

El atributo `orders` de `OrdersController` será declarado como **List<Order>**. En el constructor de `OrdersController` se creará la colección mediante `new ArrayList<>()`. También se utilizará un nuevo `ArrayList` al crear la lista temporal de resultados de la búsqueda por placa.

6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?

Los atributos serán **private** para mantener el encapsulamiento. Los constructores, getters, setters y métodos que deben ser utilizados desde otras clases serán **public**. El método `main` será **public static**.

7. ¿Qué parámetros serán requeridos por los métodos de sus clases?

Los parámetros dependerán de la operación. Por ejemplo, `addOrder` recibe un `Order`; `searchOrder` y `cancelOrder` reciben un `int orderNumber`; `updateOrder` recibe `String serviceDescription`, `double estimatedCost` e `int orderNumber`; y `searchOrdersByVehicleLicensePlate` recibe un `String vehicleLicensePlate`. Los métodos de `View` reciben los datos necesarios únicamente cuando deben mostrar información, como una `Order`, una `List<Order>`, un mensaje o un valor.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los objetos `Order` recibirán sus valores mediante su constructor al momento de registrar una nueva orden. `OrdersController` inicializará `orders` como `new ArrayList<>()`, por lo que comenzará como una lista vacía. `View` recibirá una instancia de `OrdersController` mediante su constructor e inicializará `Scanner` con `System.in`.

9. ¿Cómo identificará una orden específica dentro de la colección?

Cada orden será identificada por su **orderNumber**. El método `searchOrder(int orderNumber)` recorrerá la lista y comparará el número solicitado con `getOrderNumber()` de cada `Order`. Si existe devolverá el objeto encontrado y si no existe devolverá `null`.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Para agregar se utilizará **orders.add(order)**. Antes de registrar, `View` comprobará mediante `searchOrder()` que el número ingresado no esté repetido. Para cancelar se buscará la orden por su número y se utilizará **orders.remove(...)**.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se recorrerá `orders` con un `for-each`. Cuando `getVehicleLicensePlate().equals(vehicleLicensePlate)` sea verdadero, la orden será agregada a una lista temporal `result`. Al terminar se devolverá esa `List<Order>`; si no existen coincidencias, se devolverá la lista vacía.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

`getTotalOrdersValue()` recorrerá la colección mediante un `for-each` y acumulará `getEstimatedCost()` de cada orden. `getOrdersCostsAverage()` reutilizará ese resultado y lo dividirá entre `orders.size()`. Antes de solicitar estos cálculos, el flujo del programa verificará que existan órdenes registradas.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Se tomará como referencia inicial el costo de la primera orden y se recorrerá la lista comparando `getEstimatedCost()` de cada objeto. Cuando se encuentre un costo mayor se actualizará el valor de referencia. La operación solo se realizará cuando existan órdenes registradas.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Principalmente una entrada de tipo incorrecto, por ejemplo ingresar texto cuando `Scanner` espera un `int` o `double`, lo que puede producir **InputMismatchException**. También se considera el intento de realizar una operación con una orden inexistente. El flujo evita otros errores comprobando previamente si existen órdenes antes de utilizar operaciones que requieren elementos en la lista.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

Las entradas numéricas de `View` estarán protegidas con `try-catch` para capturar `InputMismatchException`. También se protegerá al menos una operación sobre una orden inexistente, lanzando y capturando una excepción para informar el error sin finalizar el programa. Después del manejo, el usuario podrá continuar utilizando el menú.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

Se utilizará **sc.nextLine()** dentro de `finally` después de lecturas numéricas con `Scanner`. Esto permite consumir el contenido restante de la línea tanto cuando la lectura fue correcta como cuando ocurrió `InputMismatchException`. De esta manera `Scanner` queda preparado para la siguiente entrada independientemente del resultado de la operación.